Bài thực hành 1: Python

Văn Minh Khoa

1 Đồ thị tính toán

1.1 Ý tưởng và ứng dụng

Một **đồ thị tính toán** (*computation graph*) biểu diễn một phép tính bằng các giá trị và quan hệ phụ thuộc giữa chúng. Thay vì chỉ lưu kết quả, ta còn ghi lại kết quả ấy được tạo ra từ những input nào và bằng phép toán gì.

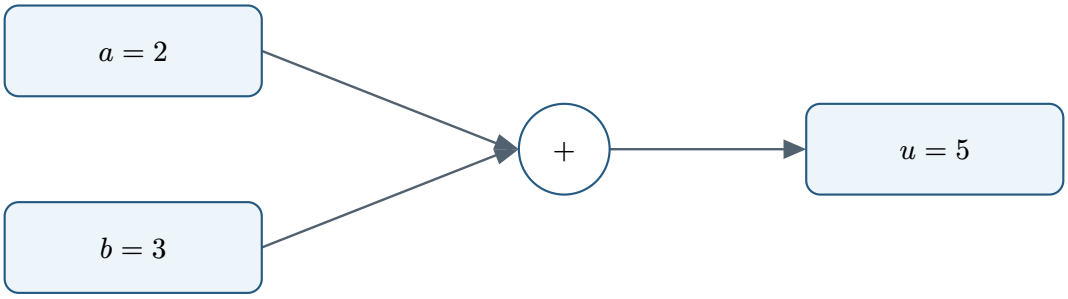
Trong các thư viện học máy, thông tin này được dùng để tính đạo hàm tự động. Khi huấn luyện mô hình, ta cần biết hàm mất mát thay đổi thế nào theo từng tham số; đồ thị tính toán giúp tổ chức việc tính các đạo hàm ấy.

1.2 Ví dụ

Xét phép tính:

$$a = 2, \quad b = 3, \quad u = a + b = 5.$$

Ta biểu diễn phép tính này bằng đồ thị sau:



Hình 1: Hai giá trị input đi qua phép cộng để tạo ra output.

Trong hình, ô chữ nhật biểu diễn một giá trị; vòng tròn biểu diễn phép toán. Mũi tên đi theo chiều **input → phép toán → output**.

Ở ví dụ này:

- a và b là hai input trực tiếp của phép cộng.
- u là output của phép cộng.
- Nếu tiếp tục tính $y = u \times 4$, thì u lại trở thành input của phép nhân tiếp theo.

Vì vậy, "input" và "output" được hiểu theo từng phép toán: một giá trị trung gian có thể là output của phép toán này, đồng thời là input của phép toán khác.

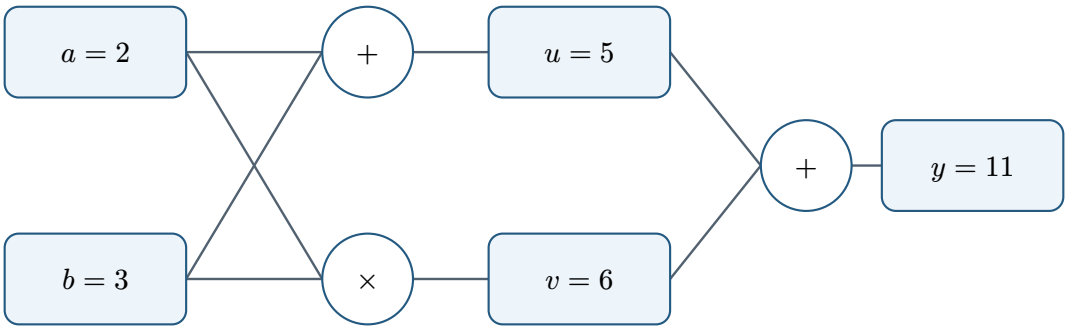
Truyền xuôi (forward pass) là sự tính kết quả các output dựa trên giá trị của input. Như trong ví dụ này, ta đi từ a và b để tính ra u.

Truyền ngược (backward pass) là sự tính đạo hàm của output theo input. Khi huấn luyện mô hình, ta cần biết hàm mất mát thay đổi thế nào theo từng tham số. Để làm được điều này, ta đi từ output về input, tính đạo hàm của output theo input. Cụ thể trong ví dụ này, ta có $u = a + b$, vậy $\frac{\partial u}{\partial a} = 1$, $\frac{\partial u}{\partial b} = 1$.

Đồ thị tính toán đặc biệt hiệu quả khi chúng ta phải truyền ngược qua nhiều phép toán. Về bản chất toán học, chúng ta chỉ sử dụng quy tắc đạo hàm hàm hợp, còn gọi là quy tắc chuỗi (chain rule). Cấu trúc đồ thị giúp ta tổ chức việc tính đạo hàm một cách hiệu quả, tránh lặp lại các phép tính trung gian.

Ta hãy xét một ví dụ phức tạp hơn: $a = 2, b = 3, u = a + b = 5, v = a \times b = 6, y = u + v = 11$.

Truyền xuôi đã khá hiển nhiên, ta hãy xem cách truyền ngược.



Hình 2: Đồ thị tính toán của $y = a + b + ab$.

Dĩ nhiên việc tính tay đạo hàm của y theo a và b là khá đơn giản. Nhưng nếu đồ thị tính toán có hàng trăm phép toán, ta mới thấy được lợi ích của việc tổ chức các phép tính theo đồ thị. Khi truyền ngược, ta chỉ cần đi từ output về input, tính đạo hàm của output theo input, và nhờ cấu trúc đồ thị, ta tránh được việc lặp lại các phép tính trung gian. Cụ thể:

- Truyền từ y ngược về các input, hay đỉnh cha của nó: $\frac{\partial y}{\partial u} = 1$ và $\frac{\partial y}{\partial v} = 1$
- Truyền từ u về các input, hay đỉnh cha của nó: $\frac{\partial u}{\partial a} = 1$ và $\frac{\partial u}{\partial b} = 1$
- Truyền từ v về các input, hay đỉnh cha của nó: $\frac{\partial v}{\partial a} = b = 3$ và $\frac{\partial v}{\partial b} = a = 2$
- Từ đó, ta có thể tính đạo hàm của y theo a và b: $\frac{\partial y}{\partial a} = \frac{\partial y}{\partial u} \times \frac{\partial u}{\partial a} + \frac{\partial y}{\partial v} \times \frac{\partial v}{\partial a} = 1 \times 1 + 1 \times 3 = 4$ và $\frac{\partial y}{\partial b} = \frac{\partial y}{\partial u} \times \frac{\partial u}{\partial b} + \frac{\partial y}{\partial v} \times \frac{\partial v}{\partial b} = 1 \times 1 + 1 \times 2 = 3$.

2 Bài tập

Hoàn thành các yêu cầu trong notebook đi kèm. Chạy các hàm test để kiểm tra tính chính xác.

Chúng ta muốn xây dựng một đồ thị tính toán. Bước đầu tiên ta phải xây dựng một class `Value` biểu diễn bởi một đỉnh. Nhìn vào ví dụ của ta, `Value` có thể là input hoặc output hoặc một toán tử (operator). Ta tổng quát hoá các thuộc tính của một đối tượng `Value` như sau:

- `data`: giá trị số thực của đối tượng.
- `label`: tên nếu có
- `op`: tức toán tử (operator)
- `parents`: các input của phép toán tạo ra `Value` đó, nếu có
- `grad`: đạo hàm của đỉnh đích, tức output sau cùng, theo đỉnh hiện tại
- `local_grads`: đạo hàm của đỉnh hiện tại, theo các input.

Theo thiết kế này, ta thấy đồ thị minh hoạ của chúng ta chỉ phân biệt hình tròn chứa toán tử và hình chữ nhật chứa output cho dễ nhìn, thực ra toán tử + chính là `op` của `Value u`, còn `Value a` thì có thuộc tính `op` rỗng. Ta xây dựng class `Value` trong Python như sau:

```
class Value:

    def __init__(self, data, parents=(), op="", label="", local_grads=()):
    # Khởi tạo
        self.data = data
```

```
        self.parents = tuple(parents)
        self.op = op
        self.label = label
        self.grad = 0.0
        self.local_grads = tuple(local_grads)
        if len(self.parents) != len(self.local_grads):
            raise ValueError("Mỗi vị trí đầu vào cần một đạo hàm cục bộ.")

    def __repr__(self):
    # Biểu diễn class
        return f"Value(data={self.data}, grad={self.grad}, label={self.label!r})"
```

2.1 Câu 1: Implement các phép toán căn bản

Python cho phép làm toán với các kiểu `int`, `float`, … nhưng class của chúng ta thì chưa. Chúng ta phải tự lập trình các chức năng đó, hay **nạp chồng toán tử (operator overloading)**. Đọc tài liệu Python để tìm ra cách nạp chồng các toán tử tại: "https://docs.python.org/3/reference/datamodel.html#emulating-numeric-types".

Yêu cầu: Mỗi phép toán phải trả về một đối tượng `Value` mới, lưu giá trị kết quả, các input trực tiếp, phép toán đã thực hiện và các đạo hàm cục bộ tương ứng. Không thay đổi các đối tượng input.

2.1.1 Phép cộng:

Giả sử ta có 2 đối tượng:

```
a = Value(2, label="a")
b = Value(3, label="b")
```

Ta cần tính `a + b` thì phải làm sao? Hãy viết phương thức (method) cần thiết cho class `Value`. Gợi ý:

```
def __add__(self, other):
```

`self` là chỉ chính đối tượng đó, `other` là một đối tượng khác, không nhất thiết thuộc class `Value`. Khi ta gọi `a + b`, thì Python sẽ hiểu là `a. __add__(b)`. Nếu `b` là kiểu `int`, `float` thì xử lí như thế nào? Nếu là một kiểu khác nữa, mà trong bài này ta không cần quan tâm, thì hãy trả về `NotImplemented`. Sau khi làm xong, hãy chạy test để kiểm tra.

2.1.2 Phép trừ:

Phép trừ cũng rất gần với phép cộng, hãy implement tương tự vô class `Value`:

```
def __sub__(self, other):
```

Sau khi làm xong, hãy chạy test để kiểm tra.

2.1.3 Phép nhân:

```
def __mul__(self, other):
```

2.1.4 Phép chia:

Anh/chị hãy đọc tài liệu tại link trên, thử xem có tìm ra cách nạp chồng phép chia không?

2.1.5 Luỹ thừa:

Anh/chị hãy đọc tài liệu tại link trên, thử xem có tìm ra cách nạp chồng phép luỹ thừa không?

Bây giờ, ta đã biết `a + b` là `a.__add__(b)`, nhưng nếu là `2 + a` thì sao?

2.1.6 Phép cộng, trừ, nhân, chia khi số hạng bên trái khác kiểu:

Đọc tài liệu tại link trên, và tìm thử xem có cách nào để implement các phép cộng, trừ, nhân, chia khi số hạng bên trái khác kiểu không?

2.2 Câu 2: Duyệt và vẽ đồ thị

Bỏ qua việc vẽ hình với Python, để vẽ một đồ thị, ta cần duyệt đồ thị, tức là duyệt các đỉnh và cạnh của nó. Class `Value` của chúng ta đã lưu mối quan hệ đỉnh-cạnh này nhờ thuộc tính `parents`. Việc của chúng ta là duyệt từ output sau cùng cho đến các đỉnh nguồn, qua việc hoàn thành hàm `trace(output)`.

Ta minh hoạ thuật toán duyệt ngay trên ví dụ cùa hình 2. Xét Hình 2, với $u = a + b$, $v = a \times b$ và $y = u + v$. y là output nơi ta bắt đầu. Hàm `trace(y)` lần theo `parents` để thu thập các node và cạnh:

1. Ghi nhận `y` và hai cạnh `u → y`, `v → y`.
2. Đi đến `u`, ghi nhận `u` và hai cạnh `a → u`, `b → u`. Tiếp tục ghi nhận `a`, `b`; cả hai không có đầu vào.
3. Đi đến `v`, ghi nhận `v` và hai cạnh `a → v`, `b → v`. Vì `a`, `b` đã được thăm, không duyệt lại hai đỉnh này.

Kết quả có **5 đỉnh và 6 cạnh**. Mỗi đối tượng chỉ được ghi nhận một lần, nhưng vẫn phải giữ đủ các cạnh.

Vậy hãy hoàn thành hàm:

```
def trace(output):
```

trong đó:

- Lưu 2 danh sách `nodes` và `edges` cho đỉnh và cạnh.
- Dùng tập hợp `visited` để lưu các đỉnh đã thăm, cho khỏi bị duyệt 1 đỉnh nhiều lần.
- Dùng cấu trúc `stack` để lưu những đỉnh chờ xử lí. Có thể viết `stack` bằng kiểu `list`, phương thức thêm vào cuối danh sách là `append()`, lấy phần tử cuối danh sách là `pop()`.
- Một cạnh (trong `edges`) có dạng `(input_node, output_node, input_index)`, trong đó `input_index` là chỉ số của đỉnh cha hay input trực tiếp trong `parents`.
- Hàm trả về `nodes` và `edges`. Không quan trọng thứ tự.

Lưu ý: mỗi đỉnh chỉ xuất hiện một lần trong `nodes`, nhưng mỗi vị trí input đều phải có một cạnh, kể cả khi hai vị trí cùng tham chiếu đến một đối tượng (như `parents=(a, a)` chẳng hạn).

2.3 Câu 3: Truyền ngược

2.3.1 Tính đạo hàm tại một đỉnh:

Tới đây, chúng ta đã có hai loại đạo hàm tại một đỉnh: `grad` tính đạo hàm của output theo biến là `Value` hiện tại, còn `local_grads` thì tính đạo hàm của `Value` hiện tại theo từng input, hay đỉnh cha. Theo quy tắc chuỗi (hay quy tắc đạo hàm hàm hợp), ta sẽ dễ dàng tính được đạo hàm output sau cùng theo các đỉnh cha của `Value` hiện tại. Đây là một bước trong sự truyền ngược, nghĩa là `grad` của 1 đỉnh sẽ được đóng góp bởi tất cả các đỉnh hậu duệ của nó. Tuy trong Python không có `private`, nhưng ta vẫn tượng trưng gọi tên method này là `_backward()`. Trong class `Value`, hãy hoàn thành method:

```
def _backward(self):
```

2.3.2 Thứ tự topo (topological order):

Thứ tự topo (topological order) là một thứ tự các đỉnh trong đồ thị sao cho mọi input (đỉnh cha) đều đứng trước ouput (đỉnh con). Ví dụ: trong hình 2, `[a, b, u, v, y]` là một thứ tự topo, `[b, a, v, u, y]` cũng là một thứ tự topo. Nhưng `[a, u, b, v, y]` thì không phải, vì b là một đỉnh cha của u.

Để xây dựng thứ tự topo của đồ thị tính toán của chúng ta, ta dùng một giải thuật đệ quy. Trước hết, ta xây dựng hàm phụ gọi là `build_topology(node, visited, order)` trong đó:

- `node` là đỉnh xuất phát
- `visited` là tập hợp các đỉnh đã thăm
- `order` là một danh sách, sẽ khởi tạo là rỗng trong hàm ngoài.
- Nếu đỉnh `node` đã thăm, ta sẽ bỏ qua (`return` không kèm theo giá trị gì cả).
- Nếu không, ta sẽ thêm đỉnh `node` vào `visited`, gọi đệ quy hàm `build_topology()` cho từng đỉnh cha của nó.
- Sau đó mới thêm `node` vào order.

Minh hoạ với hình 2: $u = a + b$, $v = a \times b$, $y = u + v$. Gọi `build_topology(y, visited, order)` với `visited` và `order` ban đầu rỗng:

1. Từ `y`, đi đến `u`, rồi đến `a`. Vì `a` không có đỉnh cha thêm `a` vào `order`.
2. Tiếp tục với `b`, thêm `b` vào `order`. Hai đỉnh cha của `u` đã được xử lí, nên thêm `u`.
3. Quay về `y`, đi đến `v`. Hai đỉnh cha `a`, `b` đã có trong `visited`, nên không xử lí lại. Thêm `v`.
4. Cuối cùng, thêm `y`.

Kết quả: `order = [a, b, u, v, y]`.

Điểm quyết định là `order.append(node)` nằm **sau** vòng lặp duyệt `parents`: các input luôn được thêm trước đỉnh sử dụng chúng. Khi truyền ngược, ta duyệt danh sách này theo thứ tự đảo lại.

Sau đó, ta sẽ xây dựng hàm ngoài `topological_order(ouput)`. Hàm này sẽ được cho sẵn vì những công đoạn quan trọng đã được anh/chị lo liệu. Nó đơn giản là:

```
def topological_order(output):
    visited = set()
    order = []
    build_topology(output, visited, order)
    return order
```

Vậy, hãy hoàn thành hàm:

```
def build_topology(node, visited, order):
```

2.3.3 Truyền ngược toàn đồ thị:

Hãy hoàn thành method sau trong class `Value`:

```
def backward(self):
```

Method này sẽ thực hiện `_backward()` trên toàn đồ thị, theo thứ tự topo đảo ngược của các đỉnh. Chúng ta đã mặc định `grad` là `0.0` trong khởi tạo đối tượng `Value`, nhưng vẫn nên gán lại cho các đỉnh cho chắc. Riêng `self.grad = 1.0` do đạo hàm một biến theo chính nó là 1.